

1. Title

Payroll Management System using Object-Oriented Programming

2. Objective

The objective of this project is to design and implement a Payroll Management System using object-oriented principles. The system aims to automate salary calculation, manage employee records, and demonstrate core OOP concepts such as abstraction, inheritance, polymorphism, encapsulation, and exception handling.

3. Problem Statement

Many small organizations face difficulties in managing payroll manually, leading to errors in salary calculation, tax deductions, and employee record maintenance. Manual systems are inefficient and prone to inconsistencies. This project provides a structured and modular solution using object-oriented programming to ensure accurate salary computation, proper employee classification, and maintainable code architecture.

4. Target Users

- Small businesses
 - Startups
 - Organizations managing payroll manually
 - Educational demonstration of OOP concepts
-

5. System Overview

The Payroll Management System allows users to:

- Add full-time and part-time employees
- Calculate salaries using different logic for each type
- Apply tax and bonus rules
- Display employee details

- Compute total payroll

The system is implemented using Java and follows a modular, class-based design.

● 6. Core Features

- Add employee (Full-time / Part-time)
 - Remove employee
 - Display employee details
 - Calculate salary dynamically
 - Compute total payroll
 - Input validation and exception handling
 - Menu-driven interface
-

● 7. OOP Concepts Used

◆ Abstraction

An abstract class `Employee` defines common attributes and behavior.

◆ Inheritance

`FullTimeEmployee` and `PartTimeEmployee` extend the `Employee` class.

◆ Polymorphism

The `calculateSalary()` method is overridden in each subclass, allowing different salary logic.

◆ Encapsulation

Private data members are accessed using getters and setters with validation.

◆ Exception Handling

Invalid inputs such as negative salary or hours are handled using exceptions.

◆ Collections

`ArrayList<Employee>` is used to store and manage employee objects dynamically.

● 8. System Architecture

The system follows a layered object-oriented architecture:

- **Employee (Abstract Class)**
Defines common attributes like ID, name, and salary.
- **FullTimeEmployee / PartTimeEmployee**
Implement specific salary calculation logic.
- **PayrollManager**
Handles employee storage, addition, removal, and payroll calculations.
- **Main Class**
Provides a menu-driven interface for user interaction.

This separation ensures modularity, scalability, and maintainability.

● 9. Salary & Tax Policy

◆ Full-Time Employees

- $\text{Salary} = \text{Base Salary} + \text{Bonus}$
- Tax applied as percentage of gross salary
- $\text{Net Salary} = \text{Gross Salary} - \text{Tax}$

◆ Part-Time Employees

- $\text{Salary} = \text{Hours Worked} \times \text{Hourly Rate}$
 - No tax applied (simplified design)
-

● 10. UML Diagram

(A UML Class Diagram is included in the docs/ folder)

The diagram shows:

- Inheritance relationships
 - Class attributes and methods
 - Interaction between components
-

● 11. Sample Output

===== PAYROLL MANAGEMENT SYSTEM =====

1. Add Full-Time Employee
 2. Add Part-Time Employee
 3. Remove Employee
 4. Display All Employees
 5. Calculate Total Payroll
 6. Exit
-

● 12. Testing & Edge Cases

Test cases handled:

- Negative salary input
 - Invalid hours worked
 - Removing non-existent employee
 - Empty employee list
-

● 13. Git Development Process

The project was developed using Git with:

- Multiple meaningful commits
- Incremental feature addition
- Refactoring and debugging stages

This demonstrates proper version control discipline.

● 14. Advantages

- Modular and scalable design
 - Clear separation of logic
 - Easy to extend (e.g., adding new employee types)
 - Demonstrates strong OOP concepts
-

● 15. Limitations

- No database integration (in-memory storage only)

- No graphical interface (CLI-based)
 - Simplified tax model
-

● 16. Future Enhancements

- Database integration (MySQL)
 - GUI using JavaFX or Swing
 - Advanced tax rules
 - Role-based access system
-

● 17. Conclusion

The Payroll Management System successfully demonstrates object-oriented design principles through a real-world application. It provides a structured approach to payroll processing and highlights the importance of modular, maintainable, and scalable software design.